BUDGE BUDGE INSTITUTE OF TECHNOLOGY, KOLKATA

SMART CALCULATION UTILITY

IT Workshop (Python) Mini Project Report

Project Information	Details
Subject	Programming with Python
Subject Code	PCC-CS393
Department	Computer Science & Engineering
Academic Year	2026–2027
Assigned Case Study	Group 8 — Reusable Utility Package for Calculations

Group Members

Name	University Roll Number
Aparupa Ray	27600125101
Upasana Ray	27600125102
Humayun Parwez	27600125103
Sudip Pal	27600125104

Prepared as documentation for the Python console application.

Abstract

Smart Calculation Utility is a menu-driven Python console application for common mathematical operations. It reduces repeated coding by organizing calculation logic into reusable functions in an importable package. Users select an operation, enter values, and receive results or clear errors. It supports arithmetic, percentage, square root, power, factorial, random integer generation, and lambda-based squaring. The project demonstrates positional/default arguments, local/global variables, built-in math and random modules, input validation, and exception handling. It runs with Python 3 in a Windows command-line terminal.

1. Introduction

Common calculations appear in many programs. Reimplementing them repeatedly duplicates logic and makes maintenance harder. This project demonstrates a modular approach by separating calculation functions from the user interface, and shows how functions and a package can be reused in other Python programs.

2. Problem Statement

Repeatedly writing code for common mathematical operations makes programs harder to maintain and reuse. This project provides reusable calculation functions behind a menu. Users choose an operation, enter values, and receive a result. The program validates several invalid inputs and displays the number of successful calculations.

3. Objectives

• Develop reusable functions for common arithmetic and mathematical operations.

• Separate calculation logic into an importable package.

• Demonstrate positional/default arguments and a lambda function.

• Demonstrate local/global variables and use math/random modules.

• Handle invalid numeric input, division by zero, negative square roots, and invalid factorial inputs.

4. Existing System

Traditional programs may implement calculation logic repeatedly or rely on standalone manual calculators. Duplication can create inconsistent validation and maintenance overhead. A manual calculator does not expose reusable functions for other Python programs. This project addresses these limits through a package and a consistent menu.

5. Proposed System

The application presents a numbered menu in main.py and routes each selection to utility_package/calculations.py. The calculation module contains reusable logic; __init__.py exposes selected functions. The program reads and validates input, displays results/errors, returns to the menu, and tracks successful calculations. No database or third-party package is needed.

6. Scope of the Project

• Addition, subtraction, multiplication, division.

• Percentage (default 10%) and power (default exponent 2).

• Square root, factorial, random integer (inclusive range), and lambda square.

• Menu handling, validation, error reporting, and success counter.

• Importable reusable Python package.

No GUI, persistent history, database, or high-precision/symbolic computation is included. The counter resets on restart.

7. Hardware and Software Requirements

Component	Requirement
Hardware	Computer/laptop, keyboard/display; ordinary resources, no special hardware.
Operating System	Windows (tested through Command Prompt).
Language	Python 3.

Component	Requirement
JDK	Not required.
IDE / Editor	Python-compatible editor; terminal for execution.
Libraries	Built-in math and random only.
Database	Not required.
Interface	Text-based menu.

8. System Design

Function-oriented modular design: main.py handles user interaction, calculations.py contains reusable functions, and __init__.py exposes package functions. No database is used, so ER/database design is not applicable.

8.1 Use Case Diagram

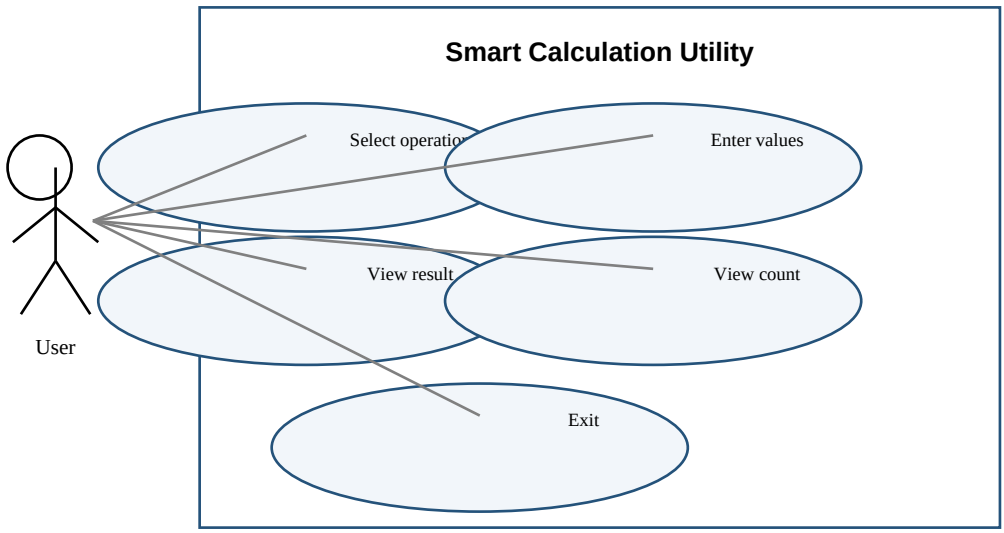


Figure 1. User interactions with the application.

8.2 Module / Class Diagram

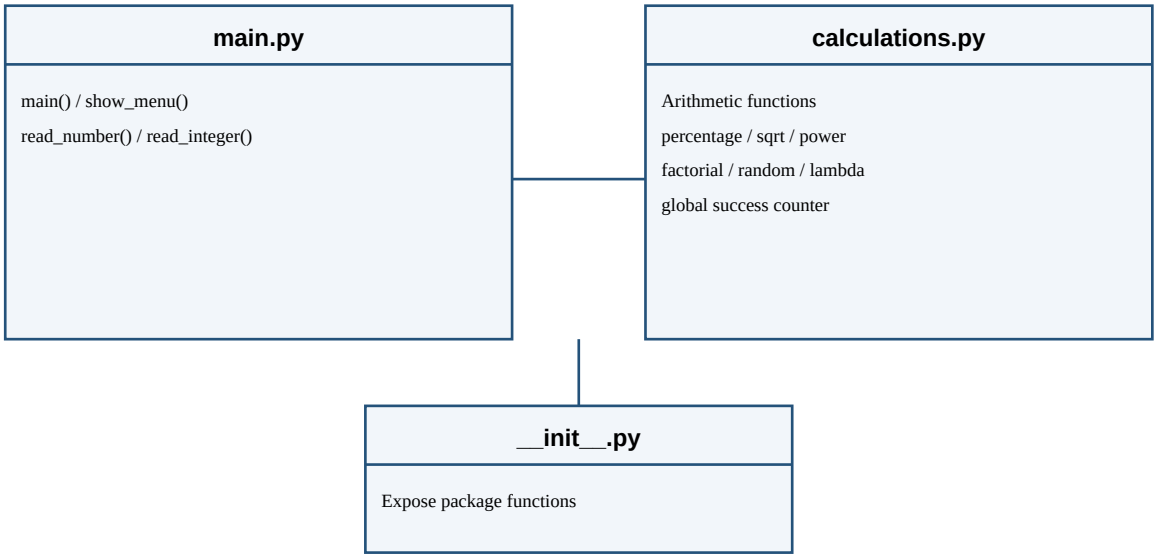


Figure 2. Package structure and responsibilities.

8.3 Activity Diagram / Flowchart

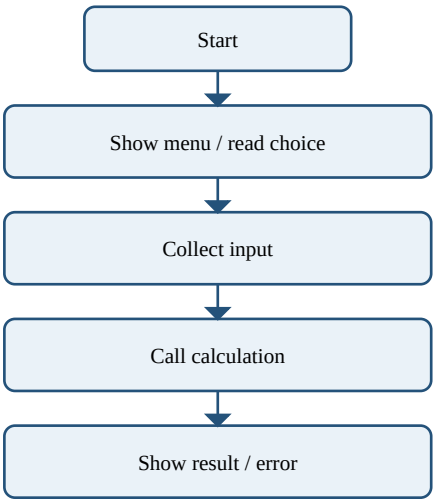


Figure 3. Repeating menu and calculation flow.

8.4 Sequence Diagram

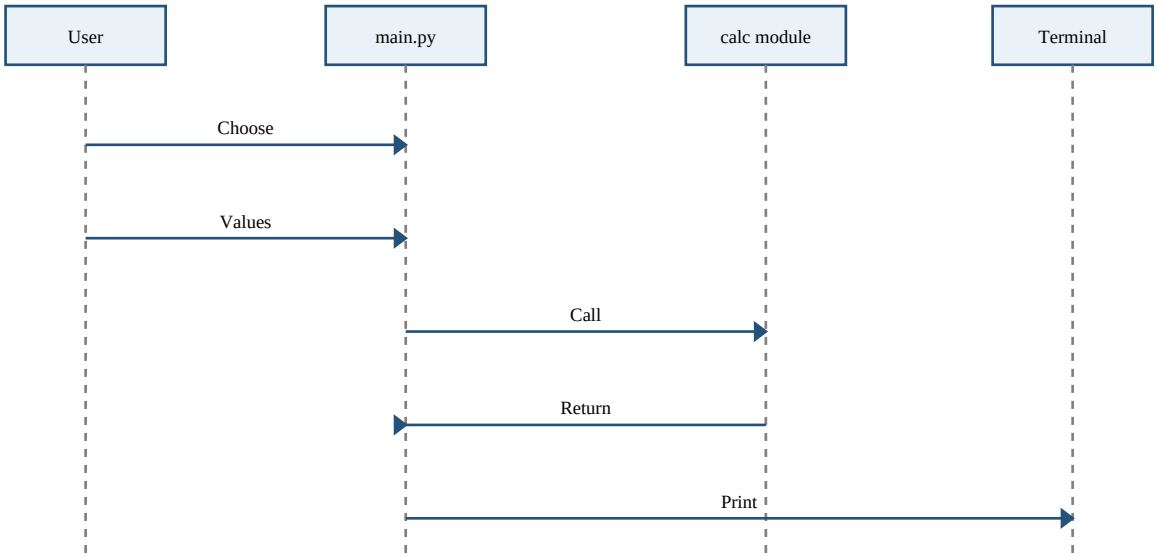


Figure 4. Typical calculation sequence.

9. Modules Description

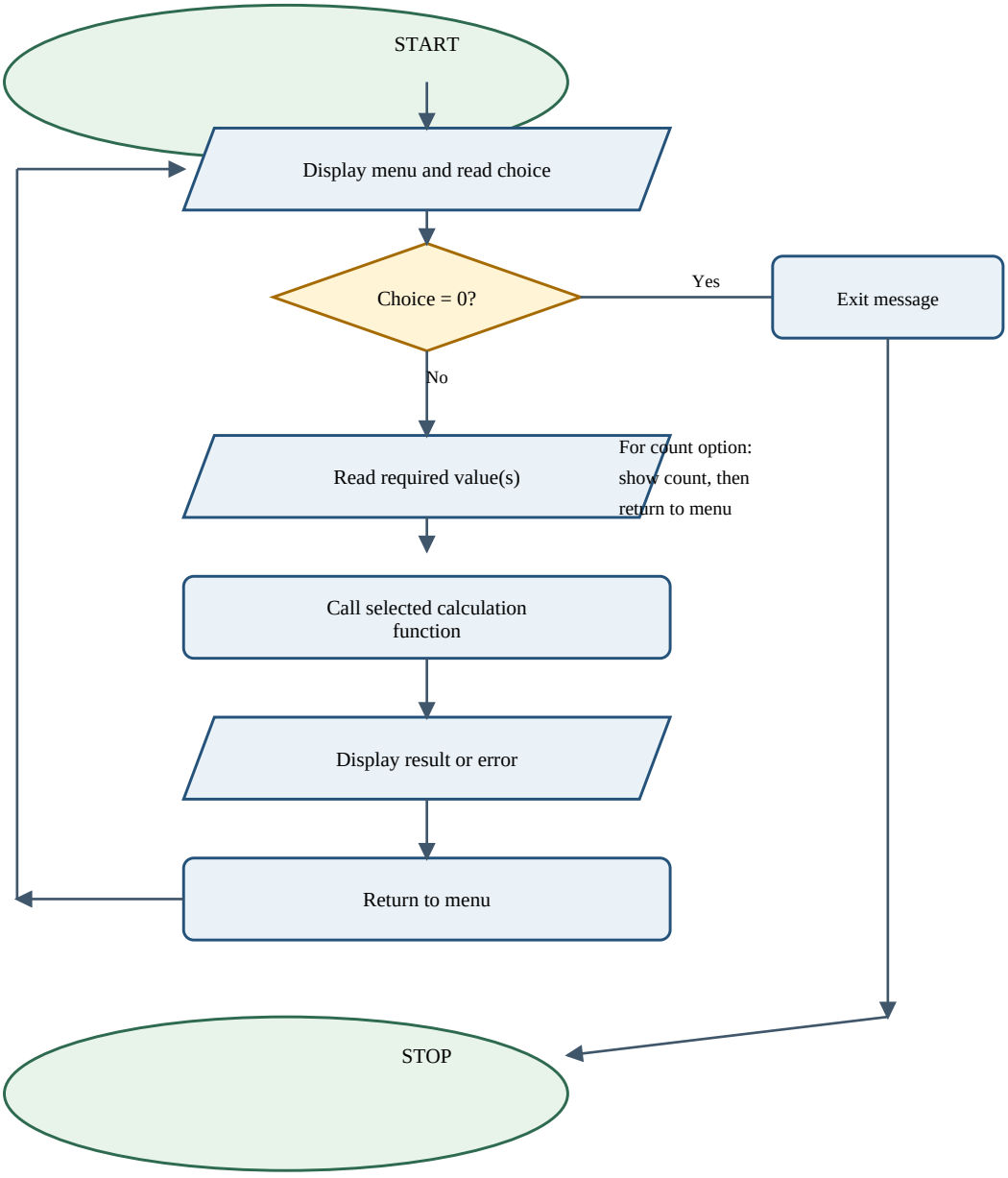
Module	Function
main.py	Displays menu, reads input, selects operations, calls functions, prints output, controls loop.
calculations.py	Reusable calculations, math/random imports, successful calculation counter.
__init__.py	Exports selected package functions.
read_number()	Reads float input and retries invalid values.
read_integer()	Reads integer input and retries invalid values.
record_calculation()	Increments counter after successful calculation.
show_menu()	Displays menu choices 0–11.

10. Algorithm and Flowchart

Algorithm for Smart Calculation Utility

1. Start the application.
2. Initialize the successful calculation counter and enter the menu loop.
3. Display the available operations and ask the user to enter a choice.
4. If the choice is **0**, display the exit message and terminate the program.
5. If the choice is **11**, display the successful calculation count and return to the menu.
6. For any valid calculation choice, prompt the user for the required number(s).
7. Validate the input. If it is invalid, display an appropriate error and return to the menu or repeat the input prompt.
8. Call the corresponding function from the calculation module.
9. Display the calculated result. If an exception occurs, display a suitable error message.
10. Return to the menu and repeat the process until the user selects **0**.
11. Stop.

Flowchart of the Main Program



Oval: Start/Stop | Parallelogram: Input/Output | Diamond: Decision

11. Implementation

Python 3 is used. main.py manages the interface while calculations.py contains the reusable operations. Representative code follows; the submitted source files are documented in the original report.

```
import math
import random

calculation_count = 0

def record_calculation():
    global calculation_count
    calculation_count += 1

def add(a, b):
    result = a + b
    record_calculation()
    return result

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero.")
    result = a / b
    record_calculation()
    return result

def percentage(number, percent=10):
    result = number * percent / 100
    record_calculation()
    return result

square = lambda number: number * number
```

ValueError is caught and displayed as an error; OverflowError is handled for excessively large results. The package uses no external libraries or database.

12. Results and Testing

The source report's terminal evidence (pages 10–17) confirms several actual outputs. For test cases without a dedicated screenshot, the result is identified as expected from the implementation, not independently evidenced.

Test	Input/action	Expected	Actual/evidence
Addition	5 and 3	8.0	Shown, source p.12

Test	Input/action	Expected	Actual/evidence
Percentage	200 at 25%	50.0	Shown, source p.11
Default percent	200; blank percent	20.0	Expected from implementation
Square root	81	9.0	Shown, source p.13
Factorial	5	120	Shown, source p.14
Lambda square	4	16.0	Shown, source p.15
Counter	Select 11	Shows count	4 shown, source p.16
Exit	Select 0	Thank-you / exit	Shown, source p.17
Divide by zero	10 / 0	Clear error	Expected from implementation
Negative sqrt	-4	Clear error	Expected from implementation
Invalid menu	Outside 0–11	Invalid message	Expected from implementation

12.1 Application Screenshots

Screenshots below are carried over from the original report and document launch/menu, percentage, addition, square root, factorial, lambda square, counter, and exit execution.

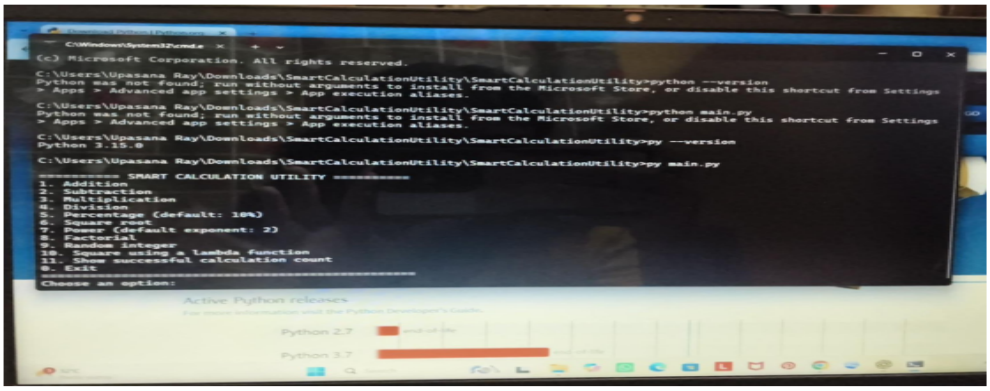
APPENDIX E — TERMINAL OUTPUT EVIDENCE

Smart Calculation Utility | Group 8 | Programming with Python

The following photographs show the project running in Windows Command Prompt using **py main.py**. They are included as evidence of execution, with the sample inputs and outputs visible in the terminal.

1. Program launch and menu

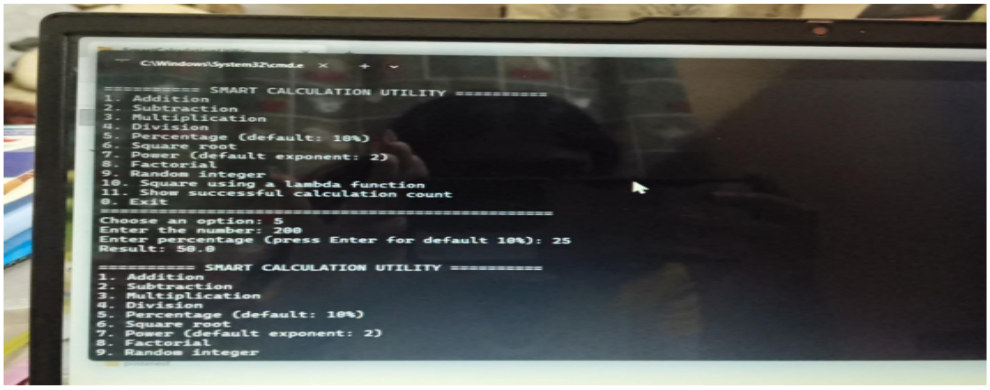
The Python launcher reports Python 3.15.0, and the project menu is displayed.



Launch/menu (source report p.10)

2. Percentage calculation

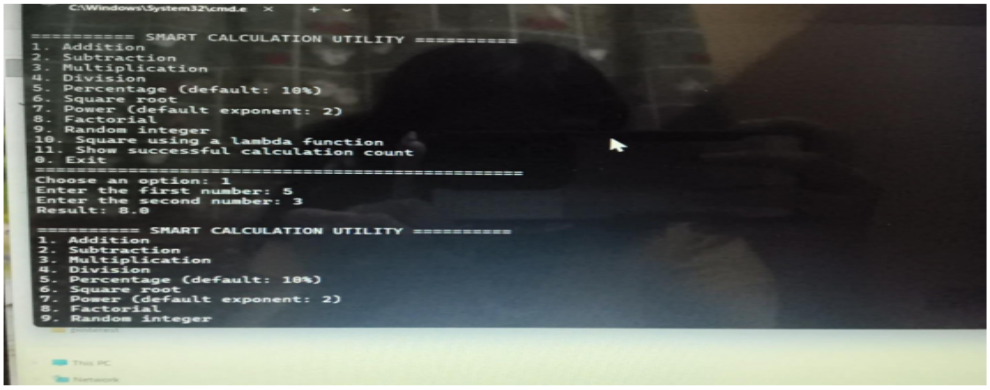
Input: 200; percentage: 25%. Output: 50.0.



Percentage result (source p.11)

3. Addition calculation

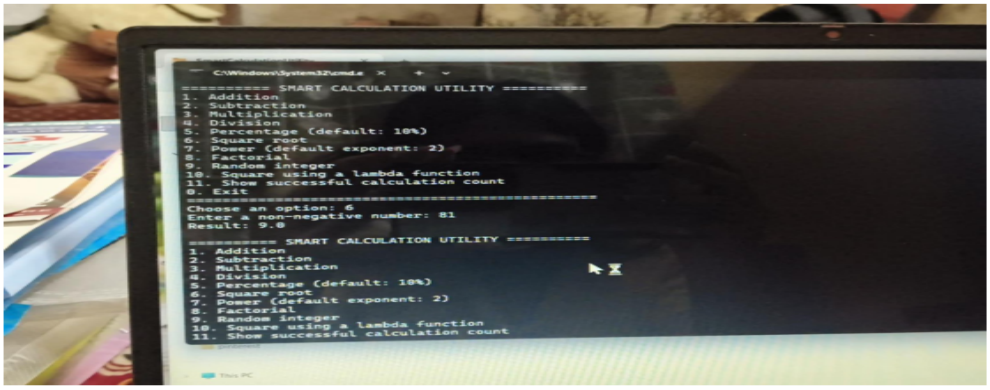
Input: 5 and 3. Output: 8.0.



Addition result (source p.12)

4. Square-root calculation

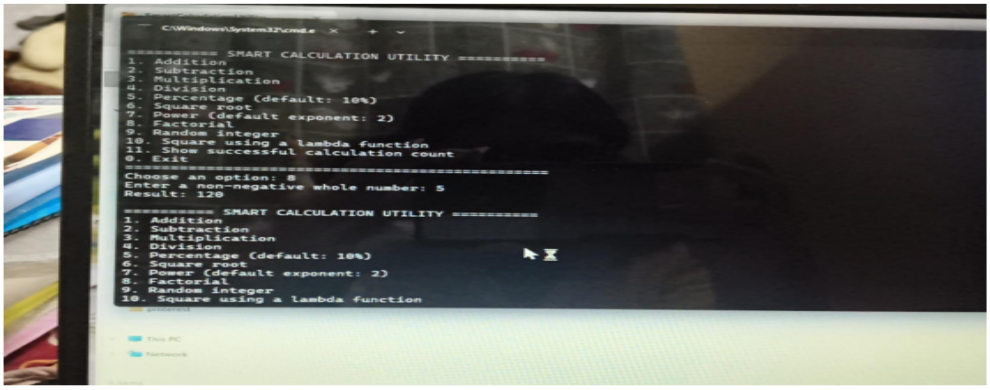
Input: 81. Output: 9.0.



Square root result (source p.13)

5. Factorial calculation

Input: 5. Output: 120.



Factorial result (source p.14)

6. Lambda-square calculation

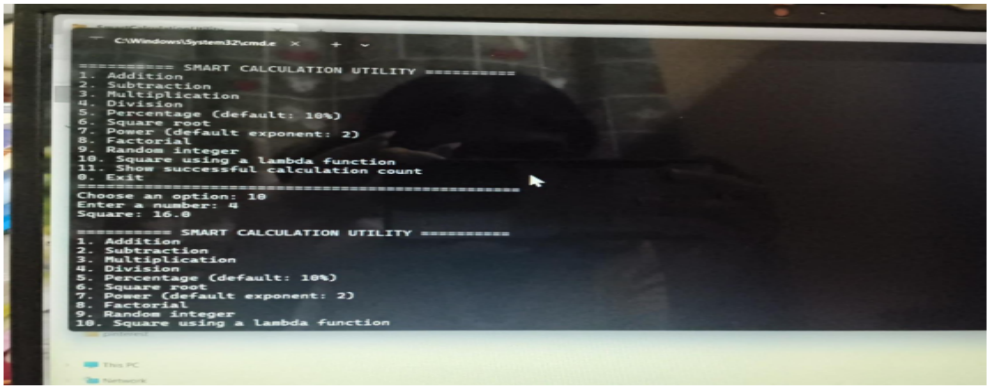
Input: 4. Output: Square = 16.0.



Lambda square (source p.15)

7. Successful-calculation counter

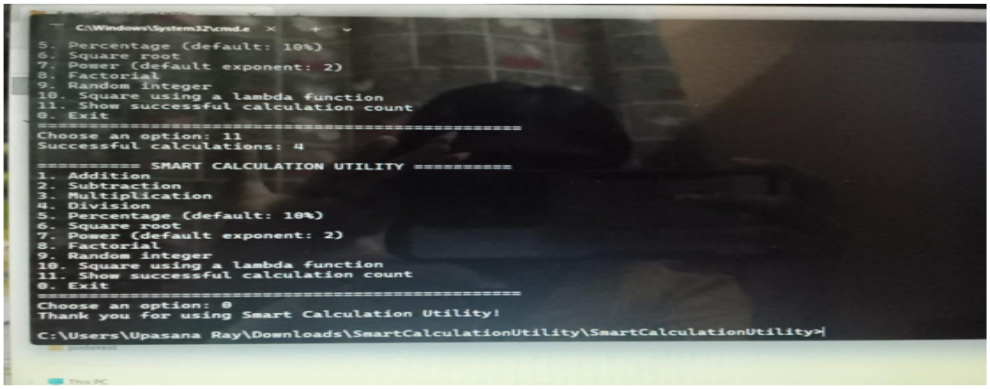
The terminal displays: Successful calculations: 4.



Successful calculation counter (source p.16)

8. Exit confirmation

The program displays its thank-you message and returns to the command prompt.



Exit confirmation (source p.17)

13. Conclusion

Smart Calculation Utility demonstrates common mathematical operations organized as reusable functions in an importable Python package. Its menu-driven interface is simple to operate, and modular design separates user interaction from calculation logic. It applies positional/default arguments, a lambda function, local/global variables, math/random modules, input validation, and exception handling. Submitted terminal evidence confirms successful execution of representative operations and the calculation counter.

14. Future Scope

• Add a graphical user interface.

• Save calculation history to a file.

• Add scientific operations and memory functions.

• Provide automated unit tests for each calculation.

• Improve input validation and edge-case reporting.

15. References

• Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/.

• Python Software Foundation, math module documentation, https://docs.python.org/3/library/math.html.

• Python Software Foundation, random module documentation, https://docs.python.org/3/library/random.html.

• Group 8, Smart Calculation Utility: IT Workshop (Python) Mini Project Report, BBIT Kolkata, Academic Year 2026–2027 (source report and terminal evidence).